RansomFlag (Flagyard) – Writeup

Debonmoy Pal

Challenge information:

Name – RansomFlag

Category – Cryptography

Goal – Recover the original flag from the provided encrypted script and its output

Initial findings:

After reading the cryptography script “crypto.py”, I figured out that it has 3 layers of encryption –

1. ChaCha20 which encrypted the original flag using a randomly generated key
2. XOR hides the key for ChaCha20 using a repeating string.
3. RSA encrypts the hidden key and prints the resulting ciphertext.

So, let’s start by decrypting each layer in the reverse order.

Given values:

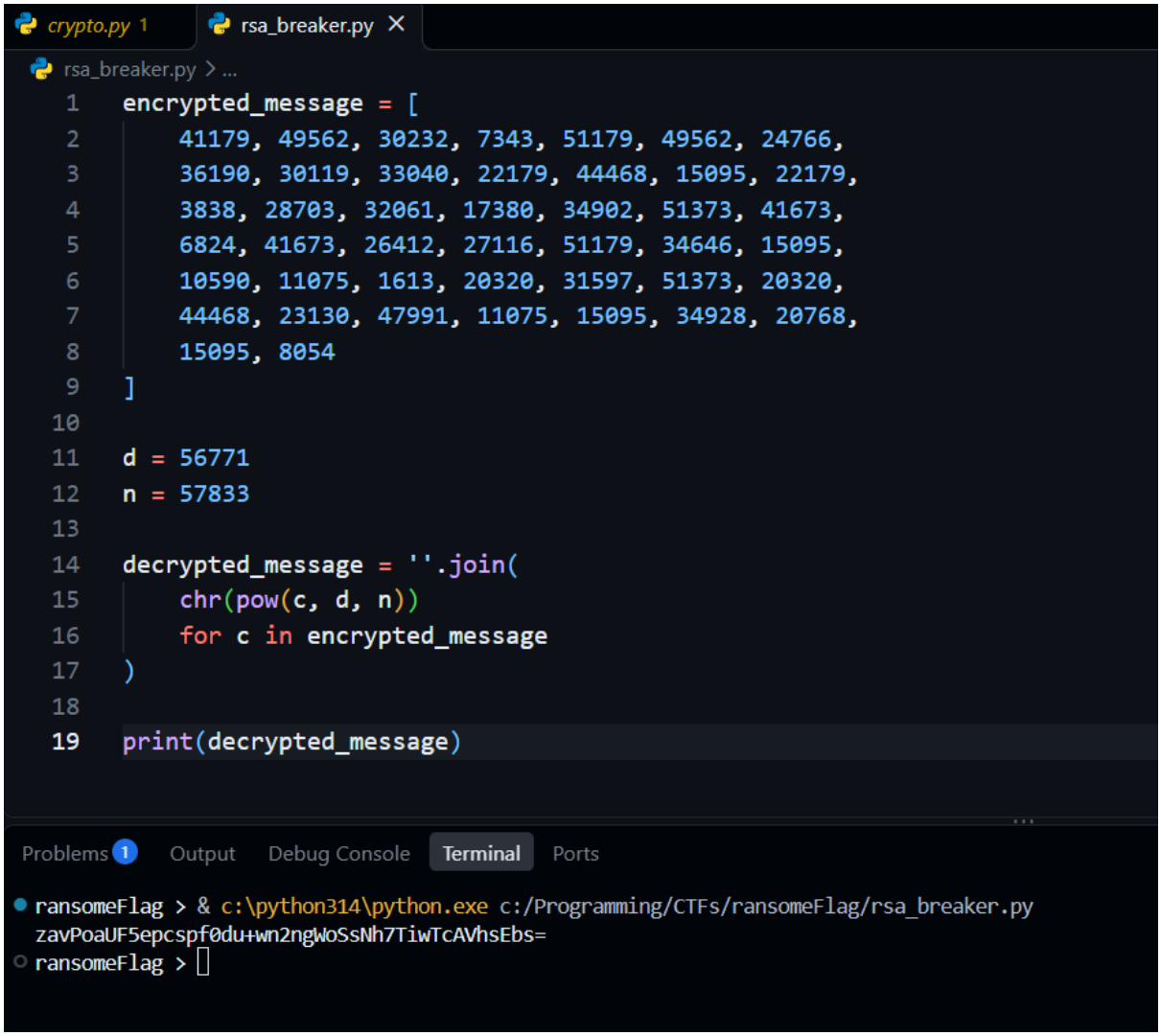
```
'''
[+] Encrypted Flag: 4HGJ/3Y6iekXR+FXdpdpa+ww4601QUtLGAzHO/8=
[+] Nonce: nFE+9jfXTKM=
[+] Private Key (d): 56771
[+] Private Key (n): 57833
[+] Encrypted Message: [41179, 49562, 30232, 7343, 51179, 49562, 24766, 36190, 30119, 33040, 22179, 44468, 15095, 22179,
3838, 28703, 32061, 17380, 34902, 51373, 41673, 6824, 41673, 26412, 27116, 51179, 34646, 15095, 10590, 11075, 1613,
20320, 31597, 51373, 20320, 44468, 23130, 47991, 11075, 15095, 34928, 20768, 15095, 8054]
'''
```

RSA layer:

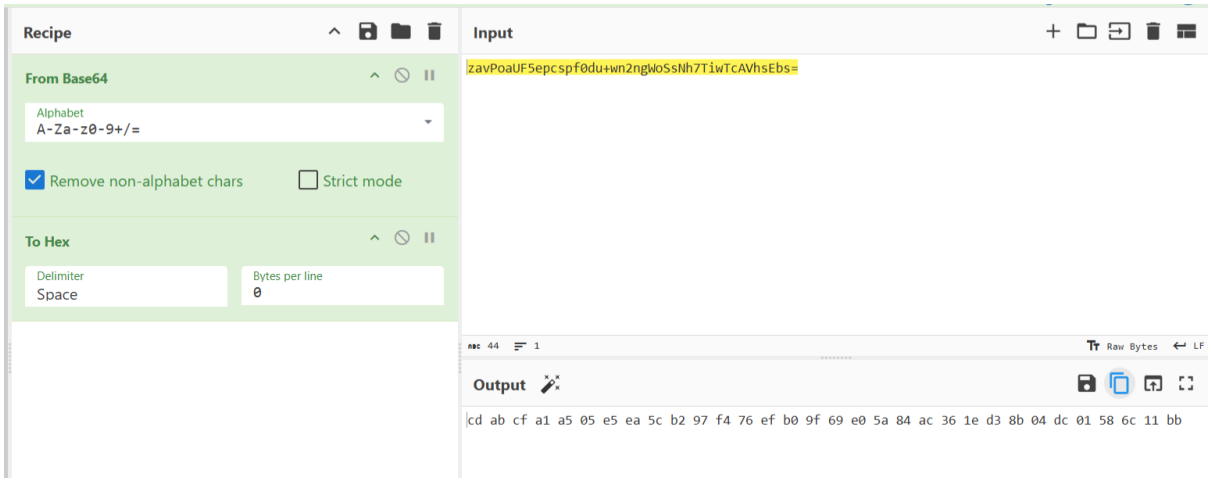
```python
# Layer 3
prime1 = randint(1, 1000)
prime2 = randint(1, 1000)
public_key, private_key = generate_key_pair(prime1, prime2, 2**4)
print(f"[+] Private Key (d): {private_key[0]}")
print(f"[+] Private Key (n): {private_key[1]}")
encrypted_message = layer3_encryption(base64.b64encode(obfuscated_key).decode('utf-8'), public_key)
print(f"[+] Encrypted Message: {encrypted_message}")
```

From this piece of code, I can figure out the rsa components required and write a simple solver script to reverse this rsa layer.

RSA Solver script:



Decrypting this in cyberchef, I get:



ChaCha20 Key Recovery:

I have the Xor key = "0x1337" from the given code file. Now, I just reverse the XOR to recover the key using this easy script:

```python
import base64

xor_key = "0x1337"

obfuscated_key = bytes.fromhex(
    "cdabcfa1a505e5ea5cb297f476efb09f69e05a84ac361ed38b04dc01586c11bb"
)

chacha_key = bytes(
    b ^ ord(xor_key[i % len(xor_key)])
    for i, b in enumerate(obfuscated_key)
)

print("Original ChaCha20 key (hex):", chacha_key.hex())
print("Original ChaCha20 key (Base64):", base64.b64encode(chacha_key).decode())
```

Output:

```
ransomeFlag > & c:\python314\python.exe c:/Programming/CTFs/ransomeFlag/chacha_key.py
Original ChaCha20 key (hex): fdd3fe929632d5926d81a4c3469781ac5ad76afc9d052de4bb7ced326b5b21c3
Original ChaCha20 key (Base64): /dP+kpYy1ZJtgaTDRpeBrFrXavydBS3ku3ztMmtbIcM=
```

ChaCha20 Decrypting and flag retrieval:

Now that I have the key for our encryption, a simple decryption script for ChaCha20 with pycryptodome solves my problem:

```python
chaDecrypt.py > ...
import base64
from chacha_key import chacha_key
from Crypto.Cipher import ChaCha20

encrypted_flag = "4HGJ/3Y6iekXR+FXdpdpa+ww4601QUtLGAzHO/8="
nonce = "nFE+9jfXTKM="

ciphertext = base64.b64decode(encrypted_flag)
nonce_bytes = base64.b64decode(nonce)

cipher = ChaCha20.new(
    key=chacha_key,
    nonce=nonce_bytes
)

plaintext = cipher.decrypt(ciphertext)

print("Decrypted bytes:", plaintext)
print("Flag:", plaintext.decode("utf-8"))
```

Output:

```
ransomeFlag > & c:\python314\python.exe c:/Programming/CTFs/ransomeFlag/chachaDecrypt.py
Original ChaCha20 key (hex): fdd3fe929632d5926d81a4c3469781ac5ad76afc9d052de4bb7ced326b5b21c3
Original ChaCha20 key (Base64): /dP+kpYy1ZJtgaTDRpeBrFrXavydBS3ku3ztMmtbIcM=
Decrypted bytes: b'FlagY{Hybr!d_Encryp7i0n_Fl4g}'
Flag: FlagY{Hybr!d_Encryp7i0n_Fl4g}
ransomeFlag >
```

Flag: FlagY{Hybr!d_Encryp7i0n_Fl4g}